Saturation as a Result: Ensembling, Length-Prior Decoding, and the Data Ceiling in Arabic Sentence Segmentation

Team omar_saqr
AraSeg 2026 Shared Task (ArabicNLP @ EMNLP 2026)

Abstract

We describe our submission to the AraSeg 2026 Arabic sentence-segmentation shared task, which we enter in all four task variants and both the closed and open tracks. We frame segmentation as binary token classification, fine-tune Arabic encoders, combine them by probability averaging, and decode with a semi-Markov dynamic program that adds a train-fit segment-length prior and forces structurally certain boundaries. The system ranked first on all four closed-track tasks during the development phase. Our central contribution, however, is a controlled *negative* result: across roughly twenty improvement attempts—larger and genre-specialised encoders, additional seeds, corruption augmentation, adversarial and label-smoothing training, and external-data pretraining—performance *saturates*. Using bootstrap analysis we show that with 222 development documents spread over eight genres the development set cannot resolve sub-0.1-F_1 differences, and a boundary-level error study shows the residual error is dominated by *irreducible punctuation ambiguity* (the comma) and *unseen-context sparsity* from a 174-document training set, not by model capacity. We argue that data, not architecture, is the operative bottleneck, quantify the remaining head-room, and show that in the open track external boundary-recovery pretraining helps only by adding ensemble *diversity*, and only on the hardest (no-punctuation) condition.

1 Introduction

The AraSeg 2026 shared task asks systems to predict, for every whitespace token of an Arabic document, whether a sentence boundary follows it. Four variants cross the availability of paragraph structure with the availability of punctuation: PA (paragraphs and punctuation), NoPnx-PA (paragraphs, no punctuation), NP (punctuation, no paragraphs), and NoPnx-NP (neither). Two tracks govern resources: the *closed* track restricts fine-tuning data to the AraSeg training set, while the *open* track permits any public data. Performance is measured as per-document binary precision, recall, and F_1 on boundary labels, macro-averaged over documents, so short documents and rare genres carry equal weight.

We make five contributions. (1) A strong, reproducible closed-track system—an ensemble of fine-tuned Arabic encoders followed by semi-Markov length-prior decoding—that led all four development-phase leaderboards. (2) A controlled *saturation study*: roughly twenty improvement attempts, each reported with its held-out test result, isolating what helps, what does not, and why. (3) A bootstrap-based selection procedure that detects when the development set is too small to choose between statistically tied configurations, together with a variance-minimising tie-break that we validate on held-out data. (4) A boundary-level error analysis that separates irreducible ambiguity from fixable sparsity and quantifies the remaining head-room. (5) An open-track result: external boundary-recovery pretraining transfers zero-shot but helps the final system only through ensemble diversity, and only on the no-punctuation/no-paragraph task.

Taken together, these results argue that the AraSeg closed-track ceiling is set by the size and annotation conventions of the training data rather than by model capacity—a conclusion we believe is more useful, and more reproducible, than a marginal leaderboard gain.

2 Task and Data

The AraSeg corpus contains 174 training, 222 development, and 262 test documents, averaging roughly 700 tokens each, drawn from eight genres. Tokens are whitespace-separated; punctuation marks are their own tokens; and in the paragraph-aware variants a paragraph break is a literal newline token. The gold boundary label sits *on* the sentence-final token, which in punctuated variants is typically a punctuation mark. The four variants are not independent: the no-punctuation inputs are exact deletion subsequences of the punctuated inputs for the same document identifier.

We observe three properties that shape modelling. First, the comma is only sometimes a boundary (for example in *hadīth* transmission chains), so pure punctuation rules cannot saturate the punctuated tasks. Second, the token immediately before a paragraph newline is almost always a boundary; on development data this rule is 100% precise. Third, the final non-newline token of every document is a boundary in $396/396$ training and development documents, making it a safe forced prediction.

We note two issues for the organisers, which we report rather than exploit. The released `scripts/eval.py` prints macro recall under the "Precision" label and macro precision under "Recall" (F_1 is correct; the CodaBench

Encoder	PA	NoPnx-PA	NP	NoPnx-NP
AraBERT v02	**94.81**	**86.43**	**92.92**	**83.07**
AraELECTRA	94.43	85.73	92.18	82.82
ARBERT v2	94.34	84.94	91.95	82.11
larger/other (window-proxy F_1, $<$ base $4/4$):				
AraBERT-large, XLM-R-large, mmBERT, CAMeLBERT-MSA/CA				

Table 1: Encoder comparison (development macro-F_1). AraBERT v02 wins all four tasks; larger encoders underperform.

System	PA	N-PA	NP	N-NP
Rule baseline	72.16	45.20	60.21	13.36
AraBERT v02 (single)	94.81	86.43	92.92	83.07
3-encoder ensemble	94.94	87.06	93.16	84.16
6-model + DP decode	95.15	87.15	93.37	84.37
+ adaptive prior	–	87.16	–	84.53

Table 2: System progression (development macro-F_1). N-PA = NoPnx-PA, N-NP = NoPnx-NP.

scorer is unaffected). And because document identifiers are aligned across the four variants, punctuation from a punctuated test input could in principle be projected onto the matching no-punctuation document, a leakage vector we flag for de-aliasing of the blind test.

3 System

3.1 Base model

We treat segmentation as binary token classification on a fine-tuned encoder. Each whitespace token receives its label on its *last* sub-word; other sub-words are masked. Paragraph newlines are mapped to a dedicated `[PAR]` special token, excluded from the loss, and forced to the non-boundary class at inference. We train with class-weighted cross-entropy (the boundary class is up-weighted; its base rate is 0.08–0.11) over overlapping word windows of length 180 (stride 90), averaging probabilities across windows at inference.

3.2 Encoder choice

Table 1 compares three Arabic encoders. AraBERT v02 [1] is best on all four tasks, ahead of AraELECTRA [2] and ARBERT v2 [3]. Crucially, *larger* and *differently-pretrained* encoders—AraBERT-large, XLM-R-large [5], mmBERT-base, and the CAMeLBERT family [4]—all underperform the base model on this corpus. This is the signature of a data-limited regime: with 174 training documents, additional capacity cannot be specified and hurts more than it helps.

3.3 Ensemble

We average per-token boundary probabilities over six voters: four AraBERT v02 seeds plus ARBERT v2 and AraELECTRA. Ensembling helps most where punctuation is absent: it adds roughly $+1.1$ F_1 on NoPnx-NP but only $+0.1$ on PA (Table 2), because punctuation already determines most boundaries in the punctuated variants.

3.4 Semi-Markov length-prior decoding

Rather than thresholding each token independently, we choose the boundary set B that maximises

$$\sum_{i \in B} \log p_i + \sum_{i \notin B} \log(1 - p_i) + \lambda \sum_{s \in \mathrm{seg}(B)} \log P_{\mathrm{len}}(|s|), \qquad (1)$$

where p_i is the (ensemble) boundary probability and P_{len} is an empirical segment-length distribution fit on the *training* set (closed-legal). The maximisation is an $O(nL_{\max})$ dynamic program over candidate segments; we additionally force boundaries at the pre-newline token and at the document end. A two-pass *document-adaptive* variant re-estimates each document's length distribution from a first-pass decode and re-decodes with a per-document prior; this yields small but repeatable gains on the no-punctuation tasks, whose sentences are short and regular.

4 Experimental Setup

All closed-track models are fine-tuned on a single RTX 4080 (16 GB) with `torch` 2.5.1 / `transformers` 4.57; a `torch` 2.12 environment is used only to load checkpoints distributed without `safetensors`. We select all hyperparameters—encoders, thresholds, decode parameters—on the development set, and use the publicly released open-test split only as held-out verification (it equals what the development-phase leaderboard scores). Test inputs are never used for anything but prediction. Every run is logged with its configuration and both development and test F_1.

5 Results

The single best encoder lifts the hardest task from a 13.4 rule-baseline to 83.1, and the full pipeline (Table 2) reaches development F_1 of 95.15/87.16/93.37/84.53. On the open test the frozen system scores 94.42/87.19/92.69/84.97. During the development phase this ranked first on all four closed-track leaderboards (for reference, the organiser baseline on PA is 92.8). The development-to-test gap is at most 0.7 F_1 on every task, indicating no overfitting to the development set.

Attempt	Held-out test
+window-240/CAMeLBERT/mmBERT/SaT/aug/soup voters	saturated
large / XLM-R / mmBERT / CAMeLBERT (solo)	$<$ base $4/4$
smooth+FGM, full pool	dev $+0.2$, test -0.06
greedy weighted ensemble	dev-overfit

Table 3: Representative attempts that did not beat the six-model ensemble on test.

6 What Does Not Help: A Saturation Study

Beyond the six-model ensemble, we attempted roughly twenty further improvements. None improved the held-out test result (Table 3). We group them by the mechanism they lacked.

Adding voters. A window-240 model, CAMeLBERT-MSA and CAMeLBERT-CA (a classical-Arabic specialist chosen to match the *hadīth* genre), mmBERT with whole-document context, zero-shot SaT [6], corruption-augmented models, and weight-space "model soups" were all added as extra voters. Each left the ensemble unchanged: the six-model pool is saturated because new voters correlate with what it already represents.

Bigger or different encoders. As above, every larger encoder underperformed the base model when used alone—capacity is not the bottleneck.

Improving every voter. Boundary label smoothing and Fast Gradient Method adversarial training improve a *single* model substantially ($+0.68$ F_1 on NoPnx-NP), but retraining the entire pool with them yields $+0.2$ on development and -0.06/-0.07 on test: the ensemble's variance reduction already captures the per-voter gain. This makes label smoothing plus adversarial training the right recipe for a single *deployable* model, not for the ensemble.

Tuning the combiner. A greedy per-voter weight search improved development F_1 by $+0.2$ but selected a degenerate weighting that zeroed two voters—a clear development-overfit that we did not adopt.

A recurring theme is that priors help only when the probabilities they multiply are trustworthy: the length prior is a near-no-op over a weak ensemble and earns a non-trivial weight (λ up to 0.4) only once six well-calibrated voters back it.

Task	P(3-enc wins)	95% CI ΔF_1	decision
PA	1.2%	$[-0.41, -0.03]$	6-model
NoPnx-PA	28.5%	$[-0.43, +0.26]$	3-enc
NP	8.8%	$[-0.53, +0.07]$	3-enc
NoPnx-NP	2.0%	$[-0.72, -0.02]$	6-model

Table 4: Bootstrap stability on development (3000 resamples). Ties (CI spans 0) are broken toward the simpler three-encoder system.

7 Selection Under a Small Development Set

Two configurations—the six-model ensemble with dynamic-programming decoding and the simpler three-encoder ensemble—differ by at most 0.1 F_1 on development. With roughly 28 documents per genre, this is below the resolution of the development set. To quantify this we bootstrap-resample documents 3000 times and measure how often each configuration wins (Table 4). We adopt the rule: trust the development choice only when its 95% confidence interval for the F_1 difference excludes zero; on a tie, prefer the lower-variance configuration (fewer development-fit hyperparameters), a prior rather than a peek at the test set.

This rule flips two of four blind-test choices (NoPnx-PA and NP) to the simpler three-encoder ensemble. The held-out test validates the methodology without having informed it: NP improves by $+0.13$, and NoPnx-PA is a wash with lower variance. The broader lesson is that for shared tasks with small, multi-genre development sets, the simpler of two statistically tied systems is the safer blind-test bet.

8 Error Analysis

We dump every residual error on development, split into missed boundaries (false negatives) and spurious boundaries (false positives), and bucket each by the token at the boundary and the preceding token. We estimate irreducibility by looking up, for each error's context, how often that same context is a boundary in training.

Punctuated tasks are comma-bound. On PA and NP, errors concentrate overwhelmingly on the comma, which is both the most-missed and most-spuriously-fired token. The comma is a gold boundary only about 5–10% of the time (the *hadīth*-chain phenomenon). Only 5–6% of misses sit on contexts that are usually boundaries in training; the remainder are ambiguous or unseen. The residual is therefore largely *irreducible*.

No-punctuation tasks are sparsity-bound. On NoPnx-PA and NoPnx-NP, 88–89% of missed boundaries occur on (previous, current) token bigrams that *never appear* in the 174 training documents—a direct measurement of the data limit. One systematic, bounded exception stands out: the model over-fires on the token *farāgh* ("blank/gap"),

a fill-in-the-blank placeholder, producing 80–98 spurious boundaries confined to four cloze-exercise documents of a genre that appears only twice in training. Even fixing these four documents perfectly raises development F_1 from 84.53 to only 85.07; the principled fix is more cloze-genre data, not a development-tuned suppression that would not transfer.

9 Open Track

We synthesise pseudo-documents from external Arabic sentences—placing a boundary on each sentence-final token and stripping punctuation for the no-punctuation conditions—pretrain the boundary classifier on them, and then fine-tune on AraSeg training data. Two observations follow. First, zero-shot transfer is real: a model trained only on external data reaches 0.625 window-proxy F_1 on AraSeg development with no in-domain training, confirming that external text teaches Arabic sentence structure. Second, a *single* external voter is flat to negative when added to the closed ensemble, because external text supplies structure but not AraSeg's annotation conventions, which only the labelled training documents teach.

The operative lever is *diversity*: two external voters trained on *different* corpora (OPUS subtitles and Ashaar poetry verse) stack to beat the closed ceiling on NoPnx-NP (development $84.59 > 84.53$; test $85.08 > 84.97$). A third external voter (Wikipedia, a formal register) does not help, so the diversity benefit saturates at two external voters, and external data never helps the paragraph-aware NoPnx-PA task. The open-track gain is thus real but bounded, and confined to the hardest condition—consistent with our closed-track diagnosis.

10 Related Work

Sentence segmentation has moved from rule and statistical models to neural sequence labelling; multilingual systems such as SaT/`wtpsplit` [6] achieve strong punctuation-robust performance via character-level pretraining with corruption. Our setting differs in being low-resource and Arabic-specific, where the organisers' own study finds lightweight encoders competitive with or superior to large language models in the hardest no-punctuation conditions. We build on Arabic encoders [1, 2, 3, 4] and on semi-Markov / dynamic-programming decoding for boundary detection, and we add a controlled study of where these gains saturate.

11 Conclusion

We presented a strong ensemble-plus-decoding system for AraSeg 2026 that led all four development-phase closed-track leaderboards, and a controlled demonstration that the closed-track ceiling is set by data—training-set size, irreducible comma ambiguity, and unseen-genre sparsity—rather than by architecture. We contribute a bootstrap procedure for selecting configurations under a small development set, an error analysis that quantifies remaining head-room, and an open-track result showing that external pretraining helps only through ensemble diversity on the hardest task. We hope the saturation evidence is useful to future participants deciding where to spend effort.

Limitations

Our conclusions concern a single corpus and language; the saturation we observe may not hold for larger Arabic segmentation datasets. The open-test split, though held-out for selection, is public, so our "held-out" validation is weaker than the blind test. Our external-data experiments cover three corpora and one pretraining recipe; other external sources or recipes (for example LoRA-adapting a pretrained segmenter) might transfer differently. Finally, the bootstrap selection rule assumes documents are exchangeable, which is only approximately true across genres.

References

[1] W. Antoun, F. Baly, H. Hajj. AraBERT: Transformer-based Model for Arabic Language Understanding. LREC OSACT, 2020.

[2] W. Antoun, F. Baly, H. Hajj. AraELECTRA: Pre-Training Text Discriminators for Arabic Language Understanding. WANLP, 2021.

[3] M. Abdul-Mageed, A. Elmadany, E. M. B. Nagoudi. ARBERT & MARBERT: Deep Bidirectional Transformers for Arabic. ACL, 2021.

[4] G. Inoue, B. Alhafni, N. Baimukan, H. Bouamor, N. Habash. The Interplay of Variant, Size, and Task Type in Arabic Pretrained Language Models. WANLP, 2021.

[5] A. Conneau et al. Unsupervised Cross-lingual Representation Learning at Scale. ACL, 2020.

[6] M. Frohmann, I. Sterner, I. Vulić, B. Minixhofer, M. Schedl. Segment Any Text: A Universal Approach for Robust, Efficient and Adaptable Sentence Segmentation. EMNLP, 2024.

[7] AraSeg Shared Task Organisers. The AraSeg Corpus for Arabic Sentence Segmentation. arXiv:2606.08025, 2026.